

Rapport de projet recherche

Aide au diagnostic de la maladie d'Alzheimer par une méthode de graph matching

Ecole Centrale de Marseille
Institut Fresnel (Groupe Signaux Multidimensionnels)

Stagiaire : Michel Pohl

Tuteur : Mouloud Adel

Mars 2016

Remerciements

Je remercie M Adel pour m'avoir accepté au sein de son équipe, m'avoir encadré au long du projet et pour le cours d'imagerie médicale qui m'a été utile lors de celui-ci.

Je remercie Yassine Zniyed, Kaoutar Abdelalim et Ibrahim Ardi, avec qui j'ai travaillé au début du projet, pour leur sympathie.

Je remercie plus généralement les membres du groupe GSM de l'équipe Fresnel, qui m'ont accueilli chaleureusement.

Je remercie M Réfrégier pour son cours sur le traitement statistique du signal, qui m'a été très utile pour ce projet également.

Je remercie M Devilez et Mme Georges pour avoir assisté à ma soutenance orale et pour les échanges intéressants que nous avons eus à ce moment-là.

Table des matières

Remerciements.....	2
Table des matières.....	3
Introduction.....	4
I – Classification par une méthode de correspondance de graphes	
1) Introduction.....	5
2) Construction des graphes de référence	
1. Normalisation des données.....	6
2. Projection des données suivant une direction $\underline{\alpha}$	7
3. Calcul des poids entre deux régions.....	7
3) Projection du graphe à tester sur un graphe de référence.....	9
4) Test du rapport de vraisemblance généralisé.....	11
5) Evaluation de l'algorithme	
1. Mécanisme d'évaluation.....	12
2. Résultats avec des choix différents de $\underline{\alpha}$	13
II – Choix optimal du paramètre $\underline{\alpha}$	
1) Fonction de coût à minimiser	
1. Position du problème.....	14
2. Probabilité d'erreur.....	14
3. Risque bayésien.....	16
4. Combinaison entre sensibilité et spécificité, aire sous la courbe ROC.....	16
5. Coût fondé sur une somme pondérée de rapports de vraisemblances	
a) Intuition et première écriture du coût.....	17
b) Introduction de coûts fixes de pondération.....	17
6. Choix de la fonction de coût utilisée.....	18
2) Restriction du domaine de définition de $\underline{\alpha}$	18
3) Méthode d'optimisation : algorithme du gradient à pas fixe.....	19
4) Structure générale du programme informatique.....	21
5) Simulations et résultats	
1. Test sans régularisation à l'infini.....	22
2. Test avec régularisation à l'infini.....	23
Conclusion.....	24
Bibliographie.....	25
Annexe : Détermination du rapport de vraisemblance.....	26

Introduction

La maladie d'Alzheimer est une maladie neurodégénérative qui touche surtout les personnes âgées et se traduit par une démence et des troubles de la mémoire. C'est un handicap socio-professionnel lourd. On estime qu'en 2050, une personne sur 85 sera affectée, or on ne dispose à ce jour d'aucun moyen de diagnostic et de traitement efficaces de la maladie.

Ce travail de 100h s'inscrit dans le cadre de la collaboration entre l'Ecole Centrale de Marseille et l'Institut Fresnel. Il est la poursuite du projet de 80h (filière recherche) que j'ai effectué avec Yassine Zniyed et Kaoutar Abdelalim. Le but fondamental est de développer une méthode de diagnostic précoce automatique de la maladie. Détecter les modifications chimiques du cerveau avant apparition des symptômes pourrait en effet permettre d'envisager un traitement avant qu'il ne soit trop tard.

On souhaite développer une méthode automatique de classification qui, à partir d'une base d'images de référence de cerveaux sains et de cerveaux malades puisse distinguer si un nouveau sujet est sain ou malade. Les classifieurs développés selon une méthode classique prennent en entrée chaque voxel des images 3D ce qui peut mener à un problème d'overfitting. Une des orientations actuelles de la recherche en ce domaine est de considérer des régions du cerveau dans leur ensemble au lieu de donner en entrée du classifieur les informations brutes de chaque voxel. Plus précisément, dans [3], Garali utilise une base d'images TEP de patients sains et malades fournie par l'Hôpital de la Timone et calcule des paramètres statistiques (moyenne, variance, asymétrie, aplatissement et entropie) sur 116 régions du cerveau identifiées par les neurologues comme des régions d'intérêt. Elle évalue l'importance de ces cinq paramètres pour la classification à partir d'un SVM standard. Dans [1], Hu et al. développent un nouveau classifieur fondé sur une méthode de graph matching sur des données monodimensionnelles pour chaque région.

Dans mon travail, à partir de la base de donnée fournie par La Timône, je propose de combiner les deux approches précédentes en étendant la méthode de graph matching de [1] à un signal qui est en fait une combinaison linéaire des cinq paramètres jugés utiles à la classification dans [3]. Dans la première partie, on montre que pour des combinaisons linéaires intuitives, la méthode donne de bons résultats. Dans la seconde, on cherche à déterminer une combinaison linéaire susceptible de donner des performances optimales au classifieur. Rétrospectivement, le poids optimal associé à chaque paramètre peut donner une information sur l'importance relative des paramètres pour la classification et donc compléter les travaux de [3].

I – Classification par une méthode de graph matching

1 °/ Introduction

Nous disposons d'une base de données issue d'images TEP cérébrales sur 61 sujets sains et 81 sujets malades. Chacune de ces 122 images 3D de cerveau est segmentée en 116 régions du cerveau qui ont été identifiées par les neurologues comme des régions d'intérêt physiologique. Pour chaque région et chaque patient, on calcule cinq paramètres : la moyenne, la variance, l'asymétrie, l'aplatissement et l'entropie des voxels correspondants. L'utilité de ces paramètres pour la classification entre patients sains et malades a été discutée dans [3]. A partir de cette base de donnée, qui se présente informatiquement sous la forme d'un cube $116 \times 5 \times 122$, nous cherchons à construire et à évaluer une méthode automatique de discrimination entre sains et malades. Nous considérons une combinaison linéaire des 5 paramètres comme détaillé dans I - 2°/ - 2. pour nous ramener à un signal monodimensionnel et appliquer la théorie développée dans [1]. Les illustrations ci-dessous sont extraites de [3].

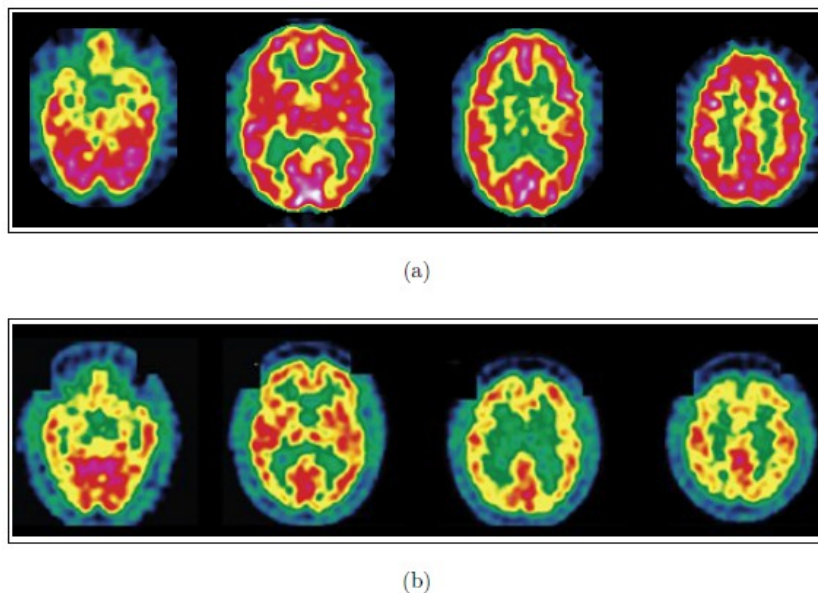


Figure 1: Coupes transversales TEP (a) patient sain (b) patient atteint de MA. La consommation de glucose d'un patient malade est réduite par rapport à celle d'un patient sain

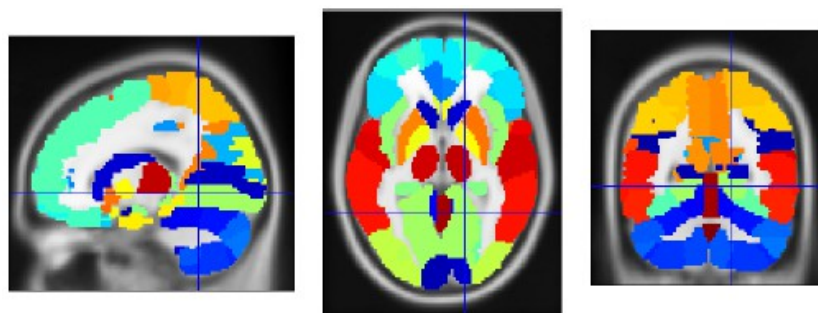


Figure 2: Segmentation anatomique en 116 régions d'intérêt

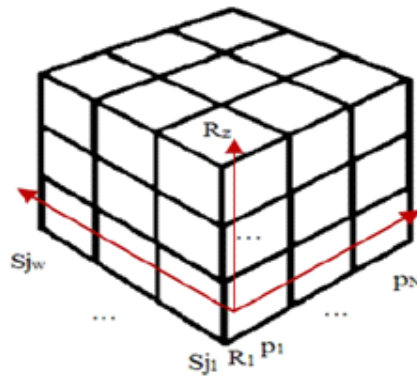


Figure 3: Représentation des données par un tenseur 3D : la première direction représente les sujets (122), la seconde les paramètres (5) et la troisième les régions (116)

2°/ Construction des graphes de référence

1. Normalisation des données

Il a été montré dans [2] que normaliser les données d'entrée permettait de mieux distinguer les sujets. C'est pourquoi la première étape de l'algorithme est de normaliser pour un paramètre fixé et un patient fixé l'ensemble des données sur la dimension des régions.

J'ai donc écrit une fonction ("function M-file") Normalisation() qui charge le tenseur des patients sains et le tenseur des patients malades (les deux se trouvant dans le répertoire de travail) et renvoie les deux tenseurs normalisés. Il s'agit de retrancher au signal sa moyenne et de diviser le tout par sa variance. Un extrait du programme se trouve ci-dessous¹.

```
%% Normalisation des données pour le tenseur des sujets sains

for i=1:S
    for j=1:5
        TenseurSain5Param(:,j,i) = (TenseurSain5Param(:,j,i) - ...
            mean(TenseurSain5Param(:,j,i)))/(sqrt(var(TenseurSain5Param(:,j,i))));
    end
end
```

¹ La variable S de l'extrait de programme représente le nombre de sujets sains et est donc normalement fixée à 61.

2. Projection des données selon une direction alpha

On veut construire deux graphes, l'un G_m à partir de l'ensemble des patients malades de la base de données, et l'autre G_s à partir des patients sains. Ils représenteront donc respectivement le cerveau "moyen" d'un sujet sain et d'un sujet malade. Les sommets de chaque graphe représentent les régions du cerveau et les poids la connectivité entre ces régions.

La première étape est de projeter le vecteur des paramètres $p_{r,s} = (p_{r,s}^1, p_{r,s}^2, p_{r,s}^3, p_{r,s}^4, p_{r,s}^5)^T$ correspondant à un sujet s et une région r selon une direction α arbitraire en calculant $p_{r,s} = \alpha^T p_{r,s}$.

```
for r=1:116
    for s=1:nb_sujets
        X(r,s) = T_norm(r,:,s)*alpha;
        % r comme "région" est s comme "sujet"
        % Attention T_norm(r,:,s) est un vecteur ligne
    end
end
```

3. Calcul des poids entre deux régions

Ensuite, on calcule le poids entre les régions i et j avec la formule :

$$W_{i,j} = \exp\left(-\sum_s (p_{i,s} - p_{j,s})^2\right)$$

où la somme porte sur le groupe de patients utilisés pour former un graphe de référence. Il s'agit de ce qu'on appelle en machine learning un noyau Gaussien.

On constate que lorsque les paramètres scalaires correspondant aux régions i et j $p_{i,s}$ et $p_{j,s}$ sont très proches pour tous les sujets s , alors la somme

$$\sum_s (p_{i,s} - p_{j,s})^2$$

est très petite et le poids résultant $W_{i,j}$ est proche de 1. En raisonnant de la même façon, on constate que lorsque les paramètres scalaires $p_{i,s}$ et $p_{j,s}$ sont très éloignés, alors la somme précédente tend vers moins l'infini et le poids résultant $W_{i,j}$ est proche de 0. En d'autres termes, lorsque les deux régions sont similaires, le poids est proche de 1 et lorsqu'elles le sont peu, le poids est presque nul. Les poids que nous calculons traduisent donc bien la notion de connectivité.

```

W = zeros(116,116);

for i=1:116
    for j=1:116
        W(i,j)=exp(-sum((X(i,:)-X(j,:)).^2));
    end
end
end

```

Les graphes G_m ou G_s représentant un groupe de sujets possèdent des propriétés particulières :

- il sont complets, c'est-à-dire que pour tous sommets i et j , il y a une arête reliant les sommets i et j .
- il sont non orientés, c'est-à-dire que si l'arête (i,j) appartient au graphe, alors l'arête (j,i) appartient également au graphe
- les arêtes ont des poids à valeurs dans l'ensemble des nombres réels R .

Nous représenterons informatiquement un graphe par sa matrice d'adjacence $\underline{W}$. L'élément d'indice (i,j) de $\underline{W}$ est égal au poids de l'arête entre les sommets i et j par définition. Le lecteur est renvoyé à [4] (chap 1) pour une introduction aux graphes.

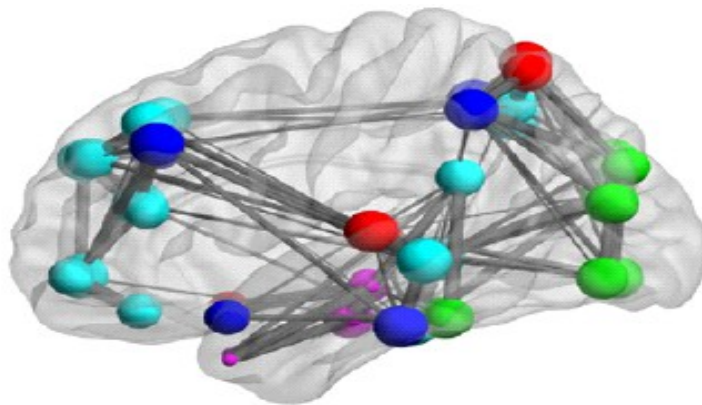


Figure 4: Représentation des régions du cerveau par un graphe. La valeur à un sommet est la combinaison linéaire des 5 paramètres statistiques de la région correspondante du sujet à tester et les poids entre deux sommets la similarité moyenne de ces deux régions obtenue sur les sujets de référence.

3°/ Projection du sujet à tester sur un graphe de référence

On appelle **signal sur le graphe** G toute application de l'ensemble des sommets de G dans l'ensemble des nombres réels $\mathbb{R}$. Il faut se représenter ce signal comme étant le sujet y dont on veut connaître la classe d'appartenance (sain ou malade). Il est maintenant nécessaire de définir quelques notions liées à l'analyse spectrale des graphes.

La **matrice des degrés** $\underline{D}$ d'un graphe non orienté à N sommets représenté par sa matrice d'adjacence $\underline{W}$ est la matrice diagonale carrée de taille N telle que :

$$D_{ii} = \sum_{j=1}^N W_{ij}.$$

La **matrice laplacienne** $\underline{L}$ de ce graphe est la matrice carrée de taille N telle que : $\underline{L} = \underline{D} - \underline{W}$.

Soit y un signal sur le graphe G . $\underline{L}$ est symétrique donc d'après le théorème spectral, on peut la diagonaliser sous la forme

$$\underline{L} = \underline{E} \underline{\Delta} \underline{E}^T$$

où $\underline{\Delta}$ est la matrice des vecteurs propres de $\underline{L}$ rangées dans l'ordre croissant, c'est-à-dire $(\underline{\Delta}_m)_{i,i} = \lambda_i$ avec $\lambda_1 \leq \dots \leq \lambda_N$.

On appelle alors **transformée de Fourier de y sur le graphe G** le vecteur $\boxed{TF(y) = \underline{E}^T y}$.

Dans le répertoire, on a créé une fonction ("function M-file") `laplace` qui prend en entrée une direction de projection α et un tenseur normalisé (un tenseur de sujets sains de référence de dimension $s \times 5 \times 116$ par exemple). `laplace` calcule d'abord pour chaque sujet i et région j du tenseur le paramètre scalaire $p_{i,j}$ défini en I -2°/ 2. Ceci permet ensuite de calculer la matrice d'adjacence $\underline{W}$ (de taille alors 116×116) du graphe moyen des sujets de référence comme expliqué dans I -2°/ 2 également. Calcule ensuite la matrice laplacienne associée au graphe représenté par $\underline{W}$ et renvoie la matrice $\underline{E}$ des vecteurs propres de $\underline{L}$. Cela permet donc par la suite de calculer aisément la transformée de Fourier d'un signal y sur les graphes G_s et G_m . On verra par la suite en quoi cette transformée de Fourier est utile pour la classification.

² Attention, la définition varie selon les références. D'ailleurs, [1] et [4] ne définissent pas la matrice des degrés de façon similaire, et l'on utilise la définition de [1] pour nos calculs.

```

% Calcul de la matrice laplacienne
L = D - W;

% Diagonalisation de la matrice laplacienne
% On a L*V = V*LAMBDA
[V,LAMBDA] = eig(L);

% Les valeurs propres et les vecteurs propres fournis ne sont pas
% nécessairement dans le bon ordre, donc il reste à les trier dans l'ordre
% croissant.

% "ind" est le vecteur des indices qui donne successivement les numéros des
% colonnes pour lequel les valeurs propres associées sont croissantes.
% eigenval est un vecteur qui ne servira pas dans la suite.
[eigenval, ind]=sort(diag(LAMBDA));

% Maintenant il suffit de permuter l'ordre des colonnes de V.
eigvec = V(:,ind);

```

Figure 5: extrait de la fonction `laplace` écrite avec Matlab

Les valeurs propres de la matrice laplacienne sont analogues aux fréquences contenues dans un signal réel et les vecteurs propres aux fonctions exponentielles complexes $t \rightarrow e^{2i\pi vt}$. Ainsi, si deux sommets i et j sont reliés par des poids élevés, les composantes d'indice i et j d'un vecteur propre $\underline{f}$ associé à une valeur propre faible³ seront proches, c'est-à-dire $\underline{f}(i) \approx \underline{f}(j)$. De même, les composantes de la transformée de Fourier d'un signal sur graphe sont analogues aux composantes spectrales d'un signal à valeur dans l'ensemble des réels⁴.

Pour un signal réel $\underline{y} = \underline{x} + \underline{b}$, une façon de se représenter le bruit $\underline{b}$ est de considérer qu'il constitue le contenu spectral associé aux fréquences élevées de $\underline{y}$. On peut alors appliquer un filtre passe-bas PB pour retrouver le signal réel $\underline{x} = PB(\underline{y})$.

Ainsi, il est raisonnable de penser que le signal réel non bruité associé au signal sur graphe $\underline{y}$ n'est constitué que par les composantes spectrales associé aux valeurs propres faibles. Plus formellement, on supposera que le signal réel est **à bande limitée**, c'est-à-dire qu'il existe un entier K tel que le signal réel $\underline{x}$ associé à l'observation $\underline{y}$ vérifie :

$$\forall k > K, TF(\underline{x})(k) = 0$$

³ On peut montrer que les valeurs propres de G_s et G_m sont nécessairement positives. (cf [7])

⁴ Cf [1] et [6] pour plus de détails à ce sujet.

4°/ Test du rapport de vraisemblance généralisé

On effectue également l'hypothèse que le bruit $\underline{b}$ associé à $\underline{y}$ est **additif, blanc, centré et gaussien**. Autrement dit, on peut écrire $\underline{y} = \underline{x} + \underline{b}$ avec $\underline{b}$ de densité de probabilité :

$$P(\underline{b}) = \frac{1}{(2\pi\sigma)^{N/2}} \exp\left(-\frac{1}{2\sigma^2} \|\underline{b}\|_2^2\right).$$

On note $L(\underline{x}_m, \sigma_m; \underline{y})$ la vraisemblance de l'observation d'un sujet malade $\underline{y}$ correspondant à un signal réel $\underline{x}_m$ sur le graphe G_b et un bruit d'écart type σ_m . De même, on note $L(\underline{x}_s, \sigma_s; \underline{y})$ la vraisemblance de l'observation d'un sujet sain $\underline{y}$ correspondant à un signal réel $\underline{x}_s$ sur le graphe G_s et un bruit d'écart type σ_s . Alors on peut montrer (cf annexe 1) que le rapport de vraisemblance généralisé qui détermine si le signal $\underline{y}$ provient d'un sujet sain ou malade est :

$$\Lambda(\underline{y}) = \frac{L(\hat{\underline{x}}_s, \hat{\sigma}_s; \underline{y})}{L(\hat{\underline{x}}_m, \hat{\sigma}_m; \underline{y})} = \left(\frac{\|\underline{y}\|_2^2 - \sum_{k=1}^K TF_m(\underline{y})(k)^2}{\|\underline{y}\|_2^2 - \sum_{k=1}^K TF_s(\underline{y})(k)^2} \right)^{N/2}$$

où N est le nombre de sommets, K l'indice défini en I - 3°/ (signal à bande limitée), TF_s l'opérateur de transformée de Fourier sur le graphe des sujets sains et TF_m celui sur le graphe des sujets malades.

On peut également montrer que

$$\Lambda(\underline{y}) = \left(\frac{\hat{\sigma}_m}{\hat{\sigma}_s} \right)^N$$

Cela indique que la classe la plus probable est celle pour laquelle on estime que le signal est le moins bruité. Autrement dit, si le signal observé du sujet à tester $\underline{y}$ sur le graphe moyen des patients sains est moins bruité que le même signal observé $\underline{y}$ sur le graphe moyen des patients malades, on estime que le sujet est sain.

Sur Matlab, on dispose d'une fonction `f_lhd_r` qui prend en entrée le vecteur alpha, le tenseur des sujets sains, le tenseur des sujets malades, le sujet à tester et l'entier K , et renvoie ce rapport de vraisemblance. Cette fonction fait naturellement appel à la fonction `laplace` que l'on a évoquée dans le paragraphe précédent.

```
% rapport de vraisemblance
% N = 116 car il y a 116 régions.
ratio = ((norme - sum((F_M*y).^2))/(norme - sum((F_S*y).^2)))^(116/2);
```

Si le rapport de vraisemblance est supérieur à 1, on considère que le patient est malade, sinon on considère qu'il est sain. Cette valeur de 1 qui semble naturelle correspond en fait à l'**hypothèse de prior non informatif** expliquée dans la partie II qui aborde probabilité d'erreur. On a donc implémenté un classifieur fondé sur la théorie du graph matching présenté dans [1] en projetant les données sur une direction arbitraire $\underline{\alpha}$.

5°/ Evaluation de l'algorithme de graph matching

1. Mécanisme d'évaluation

On effectue une boucle sur l'ensemble des 122 sujets. Pour chaque sujet sélectionné, on forme un tenseur de sujets sains et un tenseur de sujets malades avec les 121 sujets restants, bases sur lesquelles on teste le sujet que l'on a retiré. La procédure consiste donc à effectuer 122 tests et à compter le nombre de malades bien classés (VP comme vrais positifs), de sains bien classés (VN comme vrais négatifs), de malades classés sains (FN comme faux négatifs) et de sains classés malades (FP comme faux positifs). On calcule alors la sensibilité, la spécificité et la fraction de faux positifs. Cette méthode s'appelle "évaluation croisée" ou "leave-one-out cross validation" ou "1-fold cross validation".

On calcule également le score F1 donné par :

$$F1 = \frac{2 \times \text{spécificité} \times \text{sensibilité}}{\text{spécificité} + \text{sensibilité}}$$

Ce score vaut 0 lorsque la sensibilité ou la spécificité vaut 0 et atteint son maximum égal à 1 lorsque la sensibilité et la spécificité sont égaux à 1.

Decision Méthode				
Vérité		P	N	
	P	TP	FN	$\text{Sensitivity} = \frac{TP}{(TP + FN)}$ $\text{Specificity} = \frac{TN}{(TN + FP)}$
	N	FP	TN	

Figure 6: Calcul de la sensibilité et de la spécificité

Dataset	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
1	Test	Train	Train	Train	Train
2	Train	Test	Train	Train	Train
3	Train	Train	Test	Train	Train
4	Train	Train	Train	Test	Train
5	Train	Train	Train	Train	Test

Figure 7: principe des tests croisés : les valeurs de la colonne "Dataset" correspondent à l'indice k d'itération. La case encadrée dans la ligne k correspond au sujet à tester à l'itération k , et les autres cases de la ligne k aux sujets avec lesquels on construit le graphe de référence des sujets sains et le graphe de référence des sujets malades.

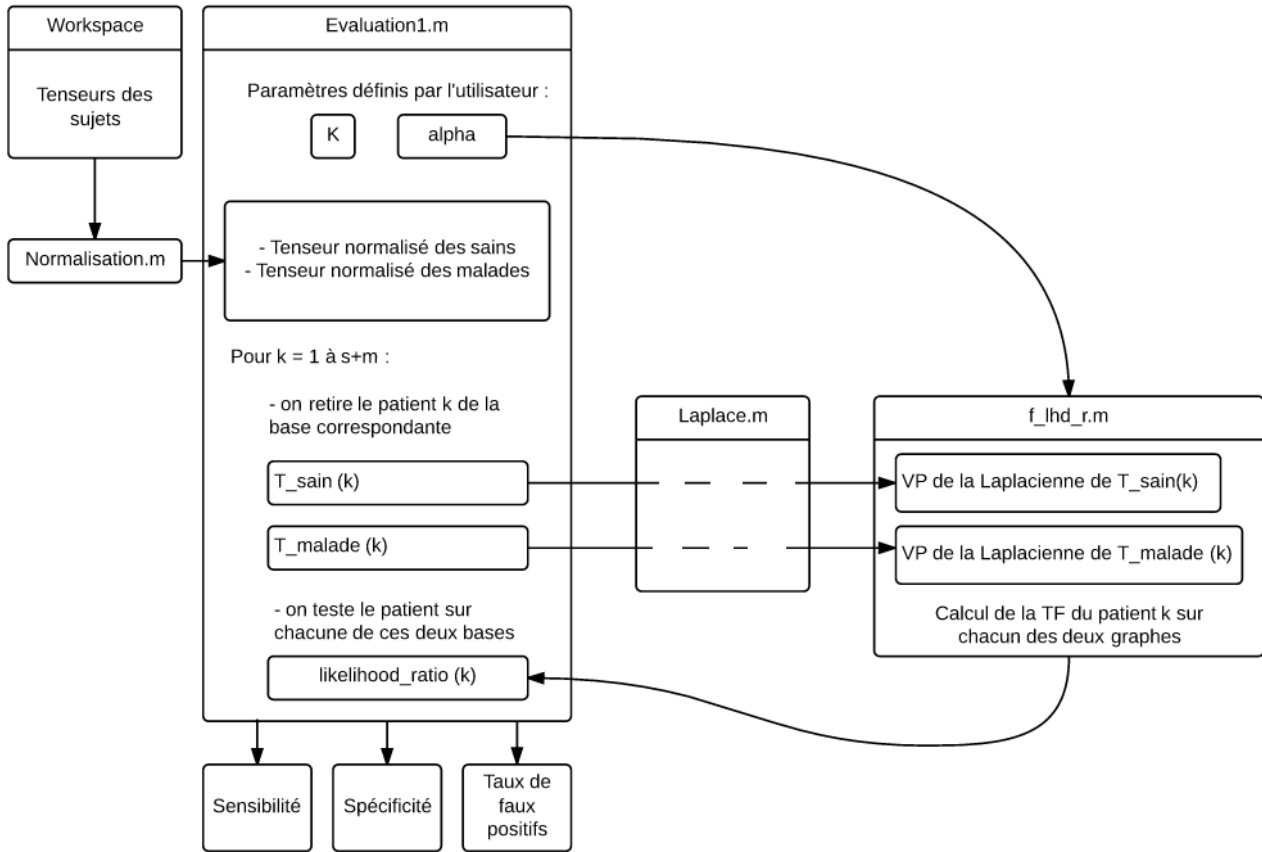


Figure 8: Structure du programme d'évaluation

J'ai alors écrit un fichier Evaluation1.m dans lequel on initialise K et α , et qui renvoie le résultat de l'évaluation croisée de l'algorithme de graph matching. Ce fichier fait donc appel à la fonction `f_lhd_r`. Dans [1] on remarque que $K = 3 \dots 5$ donne de bons résultats.

2. Résultats avec des valeurs de alpha différentes

On exécute le fichier Evaluation1.m avec différents paramètres, et les résultats sont regroupés dans le tableau suivant. Mon ordinateur (processeur intelCore i5) prend environ une minute pour donner le résultat.

	Premier test	Deuxième test
Paramètres	$K = 3^5$ $\alpha = [1;0;0;0;0]$	$K = 3$ $\alpha = [0.2;0.2;0.2;0.2;0.2]$
Résultat	Sensibilité = 0.64 Spécificité = 0.95 Taux de faux positifs ⁶ = 0.049 F1 = 0.764	Sensibilité = 0.91 Spécificité = 0.69 Taux de faux positifs = 0.31 F1 = 0.784

⁵ Nombre supposé de composantes spectrales du signal réel

⁶ Sujets vraiment sains mais diagnostiqués malades

On remarque que dans le premier cas, où seule la moyenne des pixels par région est prise en compte, le test a une forte spécificité et une faible sensibilité. C'est le cas contraire dans le deuxième cas où tous les paramètres ont le même poids. Au sens du score F1, donner la même importance à chaque paramètre améliore les performances. Il faudrait cependant tracer des courbes ROC pour analyser plus en détail les performances correspondant à chaque valeur de α car le score F1 n'est qu'une évaluation de première approche et qu'une amélioration de 0.02 n'est pas très significative. On peut également constater que la moyenne seule prise en compte donne déjà un très bon résultat, ce qui est compatible avec [3] qui montre que la moyenne joue le rôle le plus important dans la performance du résultat.

Les valeurs de α choisies pour les tests sont ici arbitraires. Cela nous amène naturellement à nous poser la question de savoir si l'on peut trouver des valeurs de α pour lesquelles le test de discrimination est plus performant, par exemple au sens du score F1. C'est l'objet de la deuxième partie de mon travail.

II – Choix optimal du paramètre α

1°/ Fonction de coût à minimiser

1. Position du problème

Dans la première partie nous avons élargi la méthode de graph matching proposée dans [1] en projetant les données provenant des 5 paramètres statistiques selon une direction α afin d'obtenir une seule grandeur scalaire par patient et par région. Nous avons effectué plusieurs tests avec des valeurs de α assez intuitives qui ont donné de bons résultats. Cependant, il serait souhaitable de savoir s'il existe une direction de projection α privilégiée qui puisse améliorer l'efficacité de l'algorithme. Si l'on dispose d'une fonction risque $R(\alpha)$ qui quantifie l'efficacité du classifieur en fonction de la valeur de α , on pourrait songer à minimiser cette fonction pour trouver le classifieur optimal au sens de ce risque.

2. Probabilité d'erreur

Une première idée serait d'utiliser la notion de probabilité d'erreur, qui dans le cas d'un test probabiliste entre deux classes, conduit au test bien connu du rapport de vraisemblance une fois minimisé. Notons R_M l'ensemble des sujets y que le classifieur classe malade et R_S l'ensemble des sujets y que le classifieur classe sain. Notons également S l'ensemble des sujets sains et M l'ensemble des sujets malades.

Alors $P(\underline{y} | S)$ est la vraisemblance de l'observation d'un signal $\underline{y}$ qui provient d'un sujet sain avec une direction de projection des données $\underline{\alpha}$ et $P(\underline{y} | M)$ est la vraisemblance de l'observation d'un signal $\underline{y}$ qui provient d'un sujet malade avec la même direction $\underline{\alpha}$. La probabilité d'erreur associée au classement est la somme de la probabilité de classer malade un patient sain et celle de classer sain un patient malade :

$$P_{err}(\underline{\alpha}, R_M, R_S) = \int_{R_M} P_{\alpha}(\underline{y}, S) d\underline{y} + \int_{R_S} P_{\alpha}(\underline{y}, M) d\underline{y}$$

Pour un $\underline{\alpha}$ donné, la minimisation de cette probabilité d'erreur donne le test du rapport de vraisemblance.

En utilisant la propriété de Bayes, on peut réécrire cette probabilité d'erreur de la façon suivante :

$$P_{err}(\underline{\alpha}, R_M, R_S) = P(S) \int_{R_M} P_{\alpha}(\underline{y} | S) d\underline{y} + P(M) \int_{R_S} P_{\alpha}(\underline{y} | M) d\underline{y}$$

On ne connaît pas la probabilité d'observer un sujet sain et celle d'observer un sujet malade. Le couple $P(S) / P(M)$ suit une loi de Bernouilli dont on peut déterminer le paramètre p empiriquement.

Mais choisir $P(S) = P(M) = 0.5$ (**hypothèse de prior non informatif**) semble une hypothèse satisfaisante. Ceci signifie que la probabilité que si un nouveau sujet est à tester, alors il a a priori autant de chance d'être sain que malade. On montre que sous cette hypothèse, pour une valeur de $\underline{\alpha}$ fixée, la **minimisation de la probabilité d'erreur** conduit au test du rapport de vraisemblance de la partie I °/4 .

Autrement dit, si $\Lambda_{\alpha}(\underline{y}) \geq 1$ alors $\underline{y}$ est classé sain.

Comme le bruit est supposé gaussien, centré, additif, blanc et le signal à bande limitée, les pseudo-vraisemblances sont égales à (cf annexe 1):

$$P_{\alpha}(\underline{y} | S) = \frac{1}{(2\pi \hat{\sigma}_s(\underline{y}, \underline{\alpha}))^{N/2}} \exp\left(-\frac{N}{2}\right) \text{ avec } \hat{\sigma}_s(\underline{y}, \underline{\alpha})^2 = \frac{1}{N} \left(\|\underline{y}\|_2^2 - \sum_{k=1}^K TF_s(\underline{y})(k)^2 \right)$$

$$P_{\alpha}(\underline{y} | M) = \frac{1}{(2\pi \hat{\sigma}_m(\underline{y}, \underline{\alpha}))^{N/2}} \exp\left(-\frac{N}{2}\right) \text{ avec } \hat{\sigma}_m(\underline{y}, \underline{\alpha})^2 = \frac{1}{N} \left(\|\underline{y}\|_2^2 - \sum_{k=1}^K TF_m(\underline{y})(k)^2 \right)$$

Pour une valeur du vecteur $\underline{\alpha}$ fixée, la minimisation de la probabilité d'erreur sous hypothèse de prior non informatif donne :

$$R_M(\underline{\alpha}) = \left\{ \underline{y} \in \mathbb{R}^N \mid \sum_{k=1}^K TF_s(\underline{y})(k)^2 - TF_m(\underline{y})(k)^2 \geq 0 \right\} \quad \text{et}$$

$$R_S(\underline{\alpha}) = \left\{ \underline{y} \in \mathbb{R}^N \mid \sum_{k=1}^K TF_m(\underline{y})(k)^2 - TF_s(\underline{y})(k)^2 \geq 0 \right\} .$$

Le vecteur alpha candidat qui améliorerait le test serait celui qui minimise la probabilité d'erreur :

$$P_{err}(\underline{\alpha}) = \frac{1}{2} \int_{R_M(\underline{\alpha})} P_{\alpha}(\underline{y} | S) d\underline{y} + \frac{1}{2} \int_{R_S(\underline{\alpha})} P_{\alpha}(\underline{y} | M) d\underline{y}$$

3. Risque bayésien

Il y a deux types d'erreurs possibles : classer sain un patient malade et classer malade un patient sain. On peut améliorer le test précédent en introduisant des coûts fixes pour pondérer une erreur par rapport à l'autre. Notons ainsi c_{SM} le coût de classer sain un patient malade et c_{MS} le coût de classer malade un patient sain. Le risque bayésien associé à ces coûts s'écrit (sous hypothèse de prior non informatif) :

$$R(\underline{\alpha}, R_M, R_S) = \frac{c_{MS}}{2} \int_{R_M} P_{\alpha}(\underline{y} | S) d\underline{y} + \frac{c_{SM}}{2} \int_{R_S} P_{\alpha}(\underline{y} | M) d\underline{y}$$

Pour une valeur de $\underline{\alpha}$ fixée, la minimisation du risque conduit au test du rapport de vraisemblance suivant : **si $\Lambda_{\underline{\alpha}}(\underline{y}) \geq c_{SM}/c_{MS}$ alors on $\underline{y}$ est classé sain.**

Autrement dit, on obtient les régions de classement :

$$R_S(\underline{\alpha}) = \{ \underline{y} \in \mathbb{R}^N \mid \Lambda_{\underline{\alpha}}(\underline{y}) \geq c_{SM}/c_{MS} \}$$

$$R_M(\underline{\alpha}) = \{ \underline{y} \in \mathbb{R}^N \mid \Lambda_{\underline{\alpha}}(\underline{y}) \leq c_{SM}/c_{MS} \}$$

Le vecteur alpha candidat est alors celui qui minimise le risque bayésien :

$$R(\underline{\alpha}) = \frac{c_{MS}}{2} \int_{R_M(\underline{\alpha})} P_{\alpha}(\underline{y} | S) d\underline{y} + \frac{c_{SM}}{2} \int_{R_S(\underline{\alpha})} P_{\alpha}(\underline{y} | M) d\underline{y}$$

4. Combinaison entre sensibilité et spécificité, aire sous la courbe ROC

Une autre idée pourrait être de chercher à minimiser le score F1 ou maximiser l'aire sous la courbe ROC obtenue après évaluation de l'algorithme.

Un problème avec cette façon de procéder est que ces fonctions calculées par le leave-one-out sur une base finie de patients ne peuvent prendre qu'un nombre fini de valeurs et ne sont donc pas suffisamment régulières par rapport à $\underline{\alpha}$ (fonctions constantes par morceaux). Par conséquent, la fonction d'optimisation risque de très mal se comporter (par exemple, si l'on utilise un algorithme de descente du gradient, comme le gradient est nul au point initial, la fonction ne sera pas minimisée).

Un autre problème est que même si l'on arrive à trouver une valeur de alpha qui optimise la nouvelle fonction, il reste d'autres valeurs de alpha pour lesquelles la fonction prend la même valeur . Autrement dit, l'extrémum global de la fonction à optimiser n'a pas un argument unique. Par conséquent, on ne sait pas, parmi les arguments alpha qui l'optimisent, lequel est le meilleur.

5. Coût fondé sur une somme pondérée de rapports de vraisemblance

a. Intuition et première écriture de la fonction de coût

Soit une valeur de α quelconque. Si pour le test d'un sujet sain y l'algorithme renvoie une valeur de rapport de vraisemblance $\Lambda(y)$ (définie en I – 4, vraisemblance du sujet sain au numérateur) très élevée, alors le test a très bien classé le sujet, donc le coût associé doit être très faible. Si le rapport est supérieur à 1 mais peu supérieur, alors le sujet est bien classé mais on a une confiance plus faible dans le résultat renvoyé, donc le coût est faible mais plus élevé que dans le cas précédent. Si le rapport est un peu inférieur à 1, le sujet est mal classé mais la confiance dans le classement est faible, donc le coût est plus élevé que dans le cas précédent. Par contre si le rapport est très proche de 0, le sujet est très mal classé, donc on voudrait pénaliser fortement le classifieur. Pour cette raison, on voudrait que le coût du test croisse avec $1/\Lambda(y)$ (si y vient d'un sujet réellement sain).

Si y vient d'un sujet réellement malade, en refaisant le raisonnement précédent, on comprend qu'il serait souhaitable que le coût du test soit croissant avec $\Lambda(y)$ (toujours avec l'expression définie en I – 4, avec la probabilité que y provienne d'un sujet sain au numérateur).

Pour prendre en compte le plus de patients possibles, on effectue la méthode du leave-one-out expliquée en I – 5°/ cette fois-ci dans la perspective d'optimiser le paramètre alpha et non plus lors du test final de l'algorithme. Plus précisément, on effectue une boucle sur les 122 patients, on retire à l'itération k le sujet k de la base de donnée, et l'on effectue un test sur le sujet k à partir des graphes issus des 121 patients restants. La fonction f_lhd_r expliquée en I-4°/ est alors utilisée pour calculer le k ème rapport de vraisemblance. Finalement, étant donné une valeur de α on trouve 122 rapports de vraisemblance issus des 122 tests de la boucle, dont on se sert pour évaluer l'algorithme.

Un choix possible pour cette fonction de coût serait alors, étant donné les remarques précédentes :

$$C(\alpha) = \sum_{y \text{ sain}} \frac{1}{\Lambda(y, \alpha)} + \sum_{y \text{ malade}} \Lambda(y, \alpha)$$

b. Introduction de coût fixes de pondération

Il a paru davantage approprié de faire apparaître des coûts fixes c_{SM} et c_{MS} représentant respectivement le coût à payer lorsque l'on classe sain un sujet vraiment malade et lorsque l'on classe malade un sujet vraiment sain. Si l'on (le médecin) pense par exemple qu'il est souhaitable de bien classer le plus possible tous les patients vraiment malades, et que l'erreur consistant à déclarer malade un patient vraiment sain est relativement peu grave par rapport à l'erreur de classer sain un patient malade, alors on peut pondérer chaque terme $\Lambda(y, \alpha)$ où y est vraiment malade par une valeur c_{SM} élevée et chaque terme $1/\Lambda(y, \alpha)$ par une valeur c_{MS} faible. Ainsi, la nouvelle fonction de coût devient :

$$C(\alpha) = c_{MS} \sum_{y \text{ sain}} \frac{1}{\Lambda(y, \alpha)} + c_{SM} \sum_{y \text{ malade}} \Lambda(y, \alpha)$$

6. Choix de la fonction de coût utilisée

Parmi tous les coûts proposés, **la minimisation du risque bayésien est clairement la plus pertinente théoriquement**, pour les raisons suivantes :

- le risque semble être une fonction continue de α
- le risque a une interprétation directe en termes de probabilités
- des coûts fixes liés aux erreurs de classification sont présents
- pour α fixé, on trouve le test le plus puissant, à savoir celui du rapport de vraisemblance.
- le risque est une grandeur calculable informatiquement

On pourrait tenter de démontrer l'efficacité de ce test de façon théorique mais ce n'est pas l'objet de mon rapport.

Pour des raisons de limitation du temps, je n'ai malheureusement pas eu le recul suffisant pour sélectionner cette méthode. Cependant, la minimisation du coût présenté en II – 1°/- 4. semble pouvoir donner des résultats intéressants.

Les coûts fondés sur la sensibilité et la spécificité ne sont pas de bons candidats pour les raisons évoquées dans le paragraphe II - 4.

J'ai donc choisi de minimiser le coût de II – 5. , qui reste intéressant malgré tout. Mais cette dernière fonction de coût possède les inconvénients suivants :

- elle n'a pas d'interprétation probabiliste.
- elle est très dépendante de la base de données de sujets utilisée.

2°/ Restriction sur le domaine de définition de α

Ou bien l'on considère que $\underline{\alpha}$ est une direction de projection, ce qui est satisfaisant intellectuellement, et l'on impose alors $\|\underline{\alpha}\|_2=1$. Ou bien l'on considère que $\underline{\alpha}$ n'a pas de signification géométrique claire et l'on peut choisir de ne pas contraindre $\underline{\alpha}$. On peut aussi imposer aux composantes de $\underline{\alpha}$ d'être toutes positives si on veut les interpréter comme des poids. Ce choix déterminera l'algorithme de minimisation utilisé (minimisation sous contraintes ou non) et le résultat final (si l'on optimise $\underline{\alpha}$ sous contraintes alors on va trouver un coût plus élevées et donc peut-être des performances plus faibles que si l'on optimise sans contraintes).

J'ai personnellement imposé $\boxed{\alpha_1 + \alpha_2 + \alpha_3 + \alpha_4 + \alpha_5 = 1}$.

J'avoue que j'avais initialement voulu fixer $\forall i, \alpha_i \geq 0$ mais qu'étant pris par le temps, j'ai fait une seconde erreur donc je n'ai gardé que la première contrainte. Cela dit, cette erreur n'a pas une influence très élevée sur mes résultats et il me semble qu'il est a priori possible d'obtenir une bonne valeur de α même avec cette contrainte.

3°/ Algorithme du gradient à pas fixe

Le problème de minimisation s'écrit alors :

$$\underline{\alpha} = \underset{\gamma \in \mathbb{R}^5 \mid \gamma_1 + \gamma_2 + \gamma_3 + \gamma_4 + \gamma_5 = 1}{\operatorname{argmin}} C(\underline{\gamma})$$

Ce qui peut également se réécrire :

$$\begin{cases} (\alpha_1, \alpha_2, \alpha_3, \alpha_4) &= \underset{\beta \in \mathbb{R}^4}{\operatorname{argmin}} C(\beta_1, \beta_2, \beta_3, \beta_4, 1 - \beta_1 - \beta_2 - \beta_3 - \beta_4) \\ \alpha_5 &= 1 - \alpha_1 - \alpha_2 - \alpha_3 - \alpha_4 \end{cases}$$

Dans la suite, à tout vecteur β dans $\mathbb{R}^4$ on fait correspondre le vecteur $\underline{\alpha} = (\beta, 1 - |\beta|)$ de façon biunivoque. Il est donc équivalent de parler de β ou de $\underline{\alpha}$.

On note $f(\beta) = C(\beta, 1 - |\beta|)$ où β est ici un vecteur de $\mathbb{R}^4$ et $|\beta| = \beta_1 + \beta_2 + \beta_3 + \beta_4$.

On se ramène ainsi au problème de la minimisation sans contraintes de f :

$$\begin{cases} \beta &= \underset{\gamma \in \mathbb{R}^4}{\operatorname{argmin}} f(\underline{\gamma}) \\ (\alpha_1, \alpha_2, \alpha_3, \alpha_4) &= \beta \\ \alpha_5 &= 1 - |\beta| \end{cases}$$

Etant donné que l'on cherche une méthode simple et que l'on a peu de temps, on utilise la méthode classique du gradient à pas fixe. Plus précisément l'algorithme d'optimisation se présente sous la forme ci-dessous, en pseudo-code ⁷ :

Algorithme du gradient

Variables à initialiser par l'utilisateur :

- pas h pour le calcul de la dérivée numérique
- nombre d'itérations maximal n_max
- norme du gradient ε en dessous de laquelle on considère que $\underline{\beta}$ est suffisamment proche du minimum de f .
- pas fixe pour la mise à jour du point courant λ
- point initial $\underline{\beta} := \underline{\beta}_0$

Initialisation des autres variables :

- norme initiale du gradient $gnorm := +\infty$
- nombre d'itérations effectuées $n = 0$
- première évaluation de f en $\underline{\beta}_0$: $f_b\grave{e}ta := f(\underline{\beta}_0)$

Boucle :

Tant que $\{ gnorm > \varepsilon \text{ et } n < n_max \}$:

```
{
   $\underline{g} := \nabla f(\underline{\beta})$  ;
   $gnorm := \|\underline{g}\|$  ;
   $\underline{\beta} := \underline{\beta} - \lambda \underline{g}$  ;
   $f\_b\grave{e}ta := f(\underline{\beta})$  ;
   $n := n + 1$  ;
}
```

Comme l'on ne dispose pas d'expression analytique pour la dérivée de la fonction de coût, on approxime celle-ci par la dérivée numérique. Plus précisément on calcule :

$$\frac{\partial f}{\partial \beta_i}(\underline{\beta}) \approx \frac{f(\underline{\beta} + \underline{h}_i) - f(\underline{\beta})}{h}$$

avec $\underline{h}_i = h \underline{e}_i$ où $\underline{e}_i$ est le i -ème vecteur de la base canonique de $\mathbb{R}^4$,

puis $\nabla f(\underline{\beta}) = \left(\frac{\partial f}{\partial \beta_i}(\underline{\beta}) \right)_{1 \leq i \leq 4}$.

Le pas h pour le calcul de la dérivée numérique doit nécessairement être plus petit que λ .

⁷ On simplifie un peu l'algorithme pour faciliter la lecture, car l'on a également sauvegardé l'historique des coûts et des valeurs de alpha. Le lecteur est renvoyé au code Matlab pour examiner les détails.

On aurait pu utiliser la formule

$$\frac{\partial f}{\partial \beta_i}(\beta) \approx \frac{f(\beta + h_i) - f(\beta - h_i)}{2h}$$

qui est plus précise mais ceci aurait nécessité d'évaluer f en plus de points par étapes de boucle (plus précisément 9 évaluations en $f(\beta)$ et $f(\beta \pm h_i)$). Or, le calcul de f en un point nécessite de calculer 121 rapports de vraisemblance (par définition du coût), donc de construire 242 graphes et de diagonaliser les 242 matrices laplaciennes différentes de taille 116×116 . On a préféré être un peu moins précis mais avoir un algorithme plus rapide (en fait utiliser optimiser le coût prend environ trois heures sur mon PC qui possède un Intel Core i5).

On peut montrer théoriquement que pour des fonctions assez régulières et pour un pas λ suffisamment petit, la suite (β_n) trouvée par l'algorithme converge vers β_{min} tel que $f(\beta_{min})$ est minimum local de f .

L'algorithme du gradient possède certains inconvénients qu'il est bon de garder en mémoire :

- nous ne sommes pas certains de trouver un minimum global de la fonction f mais seulement un minimum local.
- la convergence peut être lente si les paramètres n'ont pas le même ordre de grandeur

En pratique, on tracera les valeurs successives de la fonction de coût à minimiser pour s'assurer du bon comportement du programme d'optimisation.

4°/ Structure du programme informatique

Le programme global est constitué de deux briques importantes : la fonction `find_alpha.m` qui prend en entrée le paramètre K (nombre de valeurs propres maximales du signal réel) et renvoie le vecteur α qui minimise le coût choisi, et la fonction `Evaluation1.m` qui a été détaillée dans la partie I, qui prend en entrée α , qui évalue les performances de l'algorithme de graph matching suivant cette valeur α , et qui renvoie la sensibilité et la spécificité de ce dernier.

La fonction `find_alpha.m` fait appel à deux nouvelles fonctions : `f_cost.m` et `optimise_g`.

`f_cost.m` prend en entrée une valeur de α et renvoie en sortie le coût associé $C(\alpha)$. La structure de `f_cost.m` est presque la même que celle illustrée par le schéma de la figure 8 de I – 5°/ 1. `f_cost.m` fait en effet également appel aux fonctions `Normalisation()` et `f_lhd_r` définis dans la partie I. Les paramètres c_{MS} et c_{SM} et le paramètre de régularisation μ que l'on verra dans II – 5°/ - 2. sont définis par l'utilisateur à l'intérieur de `f_cost.m`. La fonction `optimise_g.m` prend en entrée un pointeur vers une fonction et renvoie l'argument d'un minimum local de cette dernière en suivant la méthode du gradient décrite dans II – 3°/.⁸ Les paramètres d'optimisation h , n_{max} , ε , λ et le point initial β_0 sont définis à l'intérieur de `optimise_g.m`.

⁸ Les descriptions des paramètres en entrée et en sortie sont simplifiées ici pour que la structure globale soit intelligible en première lecture. Le lecteur est renvoyé au programme informatique pour comprendre les détails.

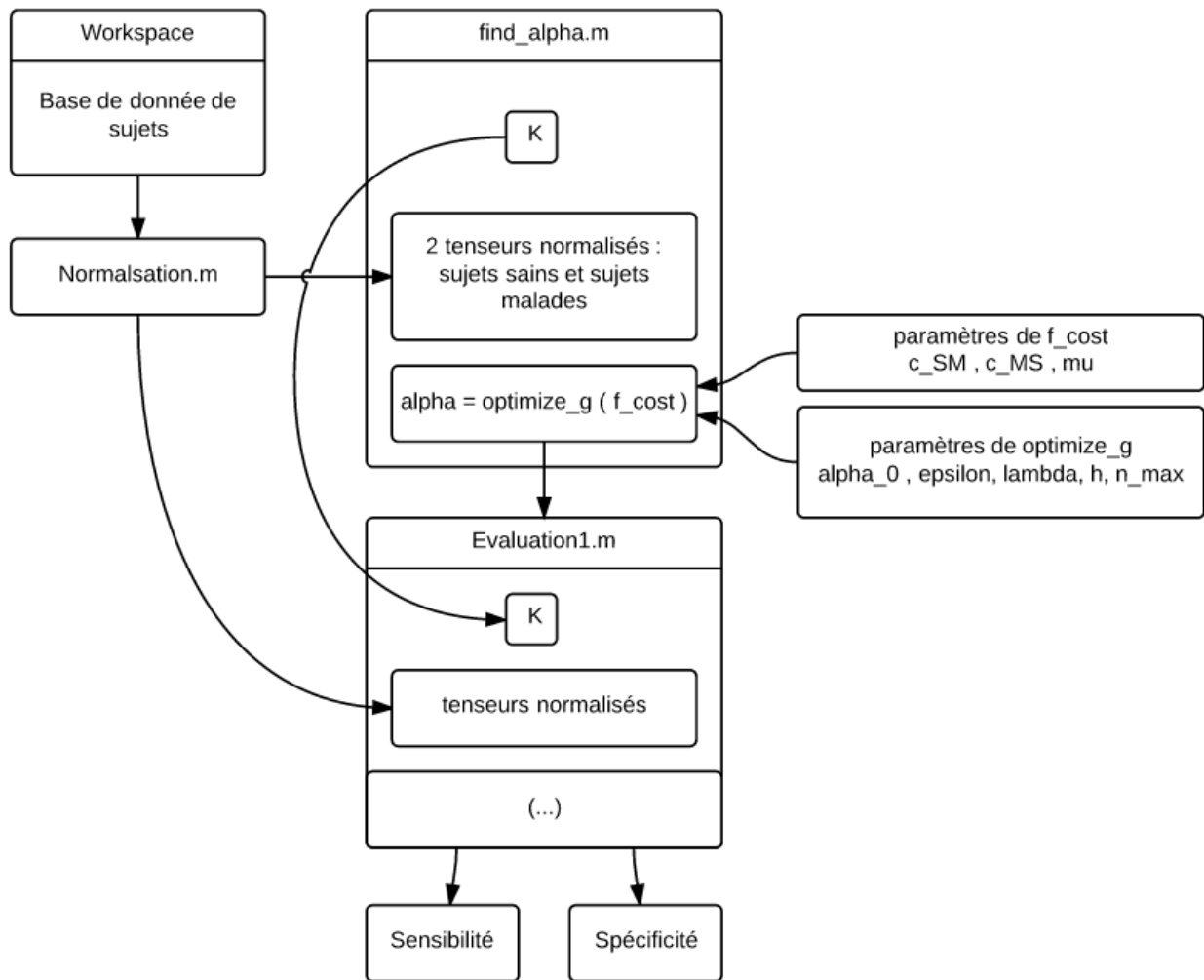


Figure 9: Structure générale simplifiée du programme

5°/ Simulations et résultats

1. Test sans régularisation en l'infini

Les paramètres utilisés sont les suivants :

- $C_{MS} = 1$
- $C_{SM} = 1$
- $K = 3$
- $h = 0.01$
- $n_{max} = 100$
- $\varepsilon = 0.001$
- $\lambda = 0.1$
- $\underline{\beta}_0 = (0.02, 0.02, 0.02, 0.02)$

L'algorithme de minimisation finit au bout de trois itérations. Le coût initial vaut $9,2 \cdot 10^6$ et il est constant au cours des deux dernières itérations, égal à 142. Les composantes du vecteur alpha sont indiquées dans le tableau suivant :

	1	2	3	4	5	6	7	8	9
1	0.2000	0.2000	0.2000	0.2000	0.2000				
2	-3.7753e+07	-2.8155e+07	6.6131e+06	-5.5230e+07	1.1453e+08				
3	-3.7753e+07	-2.8155e+07	6.6131e+06	-5.5230e+07	1.1453e+08				
4									
5									
6									
7									
8									

En réalité, l'algorithme a bien fait diminuer la fonction de coût, mais quand on exécute la fonction d'évaluation, on constate que tous les rapports de vraisemblance sont fixés à 1. La sensibilité vaut 1 et la spécificité est nulle. La valeur trouvée est extrêmement mauvaise.

Considérons en effet un graphe de référence (pour les patients sains ou malades, peu importe).

On peut écrire :

$$\begin{aligned}
 W_{i,j} &= \exp\left(-\sum_s (p_{i,s} - p_{j,s})^2\right) \\
 &= \exp\left(-\sum_s (\alpha^T p_{i,s} - \alpha^T p_{j,s})^2\right) \quad \text{donc lorsque } \|\alpha\| \text{ tend vers l'infini, } W_{i,j} \text{ tend} \\
 &= \exp\left(-\|\alpha\|_2^2 \sum_s \left(\frac{\alpha^T}{\|\alpha\|_2} p_{i,s} - \frac{\alpha^T}{\|\alpha\|_2} p_{j,s}\right)^2\right)
 \end{aligned}$$

vers 1 quels que soient i et j .

Donc toutes les régions du cerveau sont connectées de la même façon pour les sains et pour les malades, donc l'algorithme ne fait plus la différence entre les patients sains et les patients malades.

2. Test avec régularisation en l'infini

Pour remédier à ce problème, on modifie l'expression de coût en rajoutant un terme supplémentaire de régularisation à celle ci :

$$C(\alpha) = \mu \|\alpha\|_2^2 + c_{MS} \sum_{s \text{ malade}} \frac{1}{\Lambda_s(\alpha)} + c_{SM} \sum_{s \text{ sain}} \Lambda_s(\alpha)$$

μ est le paramètre de régularisation à ajuster pour empêcher $\|\alpha\|$ de tendre vers l'infini.

Pour trouver une valeur de $\underline{\alpha}$ qui convienne, on effectue des tests avec des valeurs croissantes de μ en partant au début d'une valeur relativement faible. Pour chaque valeur de μ ainsi testée, il faut régler les paramètres de la fonction d'optimisation (λ et h) afin de faire converger la suite (α_n) vers un minimum local.

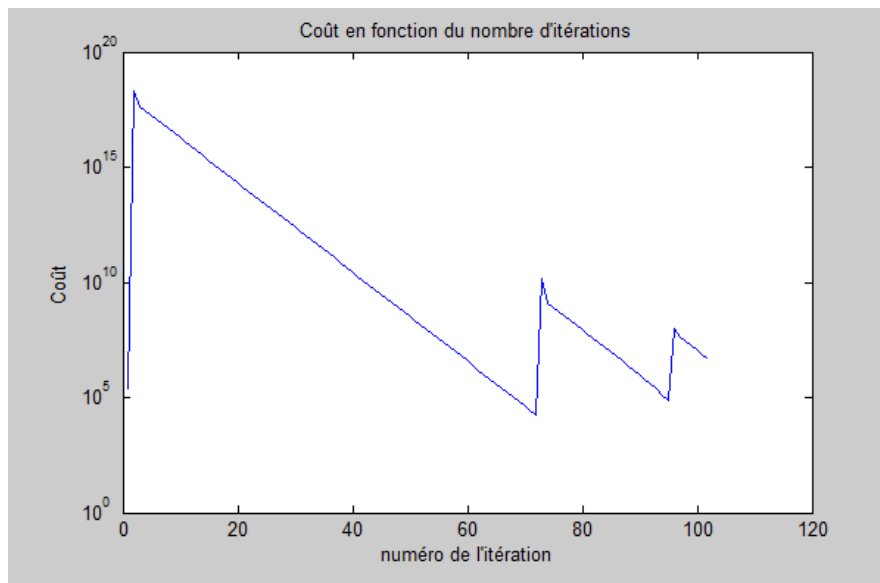
A vrai dire, je n'ai toujours pas trouvé une valeur de $\underline{\alpha}$ satisfaisante. Voici l'état des tests le jour où j'ai fini d'effectuer mon projet. Je garde toujours $C_{MS} = 1$, $C_{SM} = 1$ et $K = 3$.

J'ai effectué un dernier test avec $\mu = 100$.⁹

Les paramètres d'optimisation ont été réglés comme ci-dessous¹⁰ :

- $h = 0.0001$
- $n_{max} = 100$
- $\varepsilon = 0.001$
- $\lambda = 0.001$
- $\underline{\beta}_0 = (1, 0, 0, 0)$

Voici la représentation du coût en fonction du nombre d'itérations, en échelle logarithmique :



On constate que l'on a encore des problèmes de convergence malgré que l'on ait diminué les valeurs de h et λ puisqu'il y a des discontinuités du coût en fonction de l'itération. On peut observer que ces discontinuités correspondent à des sauts vers l'infini de $\|\underline{\alpha}\|$. On constate cependant que la 72ème valeur de $\underline{\alpha}$ juste avant le deuxième saut a un coût plus faible que le coût initial.

⁹ Pour $\mu = 1$, je me suis retrouvé dans la même situation que dans $\Pi - 5^\circ - 2$, c'est à dire que $\|\underline{\alpha}\|$ est très grand et les rapports de vraisemblance sont fixés à 1.

¹⁰ On a choisi les valeurs précédentes de h et λ après avoir constaté que pour $\lambda = 0.01$ et $h = 0.001$ il n'y avait pas convergence de la suite (α_n) .

Cependant, quand on fait appel à la fonction d'évaluation avec cette valeur de $\underline{\alpha}$, on trouve une sensibilité de 17.2% et une spécificité de 75.4% ce qui correspond à des performances faibles, et en tout cas plus faibles que celles correspondant au point initial $\underline{\alpha}_0 = (1, 0, 0, 0, 0)$.¹¹

Cependant, cela ne permet pas de conclure quant à l'efficacité de la méthode car :

- l'algorithme n'a pas fini de converger vers un minimum local
- peut-être que la valeur de μ choisie est trop élevée qu'alors le terme de régularisation dénature trop la fonction de coût.

¹¹ cf paragraphe I – 5° / - 2.

Conclusion

Ce travail poursuit la tendance actuelle à vouloir discriminer patients sains et malades à partir de paramètres globaux calculés sur des régions cérébrales d'intérêt sur d'images fonctionnelles. Dans ce travail, on a cherché à combiner l'approche de graph matching développée dans [1] et l'approche qui consiste à prendre en compte plusieurs paramètres statistiques développée dans [3]. On a pu montrer que des combinaisons linéaires simples de la moyenne, variance, aplatissement, asymétrie et entropie des régions d'intérêt donnent de bons résultats lorsqu'ils sont fournis en entrée du classifieur "graph matching", sous des hypothèses de prior non informatif, de bruit blanc centré gaussien additif, de signal à bande limitée et en utilisant des noyaux gaussiens pour mesurer la similarité des régions. On a ensuite cherché à trouver une combinaison linéaire qui puisse améliorer les performances de l'algorithme actuel. On ne peut pas conclure sur la méthode proposée, qui utilise les rapports de vraisemblance sur un test d'évaluation croisée, car je n'ai malheureusement pas pu terminer mes tests. L'autre approche fondée sur la minimisation du risque bayésien semble prometteuse et est une autre piste à poursuivre. Dans la suite des travaux, on pourrait s'intéresser à étudier des mesures de similarité autres que le noyau gaussien ainsi que des hypothèses plus complexes que celles de signal à bande limitée.

Ce travail est ma première expérience de la recherche en informatique et en mathématiques appliquées. Il m'a permis de continuer à développer ma compréhension des méthodes de traitement statistique du signal et de machine learning. J'ai également revu quelques notions en théorie des graphes et de l'optimisation qui me seront très utiles par la suite en tant que futur chercheur en informatique. C'était également l'occasion pour moi de développer un programme depuis la conception de son architecture jusqu'aux tests de chaque fonction, en temps relativement limité. Mais surtout, j'ai pu exercer ma démarche scientifique en formulant et formalisant de nouveaux problèmes, en proposant des pistes de recherche, en formulant des hypothèses, en mettant mes hypothèses à l'épreuve de tests, etc. J'espère que mon travail pourra être utile et donner de nouvelles idées dans la recherche contre les maladies neurologiques.

Bibliographie

- [1] *Matched signal detection on graphs: Theory and application to brain network classification*, C Hu, L Cheng, J Sepulcre, G El Fakhri, YM Lu, Q Li, Information Processing in Medical Imaging, 1-12
- [2] Abdelalim Kaoutar, Zniyed Yassine, Pohl Michel, *Aide au diagnostic de la maladie d'Alzheimer*, rapport de projet filière R&D, Centrale Marseille, 2016
- [3] Garali Imène, *Aide au diagnostic de la maladie d'Alzheimer par des techniques de sélection d'attributs pertinents dans des images cérébrales fonctionnelles obtenues par Tomographie par Emission de Positons au 18FDG*, Institut Fresnel, Université Aix-Marseille, thèse soutenue le 7 décembre 2015
- [4] Image processing and Analysis with Graphs, Theory and Practice, Olivier Lezoray, Leo Grady, CRC Press, 2012
- [5] Boutarfass Sanae, *Classification d'images de TEP sur la base d'une représentation par graphes*, rapport de projet de fin d'études, AMU, 2015
- [6] D. I. Shuman, S. K. Narang, P. Frossard, A. Ortega, and P. Vandergheynst, "The emerging field of signal processing on graphs," IEEE Signal Proc. Mag., vol. 30, no. 3, pp. 83–98, 2013.
- [7] Chung, F.R., 1997. Spectral graph theory. Am. Math. Soc. 92.

Annexe : détermination du rapport de vraisemblance

La démonstration qui suit provient en grande partie de [1] mais j'essaye de la réécrire pour en corriger les imprécisions auxquelles j'ai été confronté. On considère un signal $\underline{y}$ correspondant à une observation sur un sujet. On connaît le graphe des sujets sains G_s et le graphe des sujets malade G_m . On fait l'hypothèse que $\underline{y}$ contient un **bruit blanc centré additif Gaussien** et l'on cherche si le signal non perturbé provient du graphe G_s ou du graphe G_m . En d'autres termes, on écrit $\underline{y} = \underline{x} + \underline{b}$ où $\underline{b}$ est blanc gaussien, et où $\underline{x}$ est le signal réel qui provient de l'un des deux graphes G_s ou G_m . On fait de plus l'hypothèse que le signal réel est à **bande limitée**, c'est-à-dire qu'il existe un entier K tel que les composantes spectrales de $\underline{x}$ qui sont supérieures à K sont nulles.

Les deux hypothèses alternatives testées sont donc :

(H_m) $\underline{y} = \underline{x}_m + \underline{b}_m$ où $\underline{x}_m$ est un signal à bande limitée sur le graphe G_m .

(H_s) $\underline{y} = \underline{x}_s + \underline{b}_s$ où $\underline{x}_s$ est un signal à bande limitée sur le graphe G_s .

La probabilité d'observer le signal $\underline{y}$ dans l'hypothèse (H_m) est :

$$L(\underline{x}_m, \sigma_m; \underline{y}) = \frac{1}{(2\pi\sigma_m)^{N/2}} \exp\left(-\frac{1}{2\sigma_m^2} \|\underline{y} - \underline{x}_m\|_2^2\right)$$

où σ_m est la variance du bruit $\underline{b}_m$ et N le nombre de sommets du graphe.

On note $\underline{F}_m$ la matrice des vecteurs propres de la matrice laplacienne du graphe G_m .

Plus précisément, $\underline{L}_m = \underline{F}_m \underline{\Lambda}_m \underline{F}_m^T$ où $\underline{L}_m$ est la matrice laplacienne du graphe G_m et où $\underline{\Lambda}_m$ est la matrice des vecteurs propres de $\underline{L}_m$ rangées dans l'ordre croissant, c'est-à-dire $(\underline{\Lambda}_m)_{i,i} = \lambda_i$ avec $\lambda_1 \leq \dots \leq \lambda_N$.

$$\text{Alors } L(\underline{x}_m, \sigma_m; \underline{y}) = \frac{1}{(2\pi\sigma_m)^{N/2}} \exp\left(-\frac{1}{2\sigma_m^2} \|\underline{F}_m \underline{TF}_m(\underline{y}) - \underline{F}_m \underline{TF}_m(\underline{x}_m)\|_2^2\right)$$

où $\underline{TF}_m(\underline{x}_m)$ est la transformée de Fourier de $\underline{x}_m$ sur le graphe G_m

et $\underline{TF}_m(\underline{y})$ la transformée de Fourier des $\underline{y}$ sur le graphe G_m .

Comme $\underline{F}_m$ est orthonormée,

$$L(\underline{x}_m, \sigma_m; \underline{y}) = \frac{1}{(2\pi\sigma_m)^{N/2}} \exp\left(-\frac{1}{2\sigma_m^2} \|\underline{TF}_m(\underline{y}) - \underline{TF}_m(\underline{x}_m)\|_2^2\right).$$

On estime $\underline{x}_m$ selon le critère du maximum de vraisemblance (MV), ce qui donne, en vertu de l'hypothèse du signal à bande limitée :

$$\underline{TF}_m(\hat{x}_m)(k) = \begin{cases} \underline{TF}_m(\underline{y})(k) & \text{si } k \leq K \\ 0 & \text{sinon} \end{cases}$$

où K correspond à l'indice de la valeur propre associée λ_k .

De même, l'estimée de $\underline{x}_s$ selon le critère du MV s'écrit :

$$TF_m(\hat{x}_s)(k) = \begin{cases} TF_s(y)(k) & \text{si } k \leq K \\ 0 & \text{sinon} \end{cases}$$

L'estimation du signal réel correspond au signal observé filtré par un passe-bas idéal.

De même, on estime σ_m avec le critère du maximum de vraisemblance. La dérivée de la log-vraisemblance par rapport à σ_m s'écrit :

$$\frac{\partial l}{\partial \sigma_m}(\hat{x}_m, \sigma_m; \underline{y}) = -\frac{N}{\sigma_m} + \frac{1}{\sigma_m^2} \|TF_m(y) - TF_m(\hat{x}_m)\|_2^2$$

L'estimée de la variance du bruit correspond à une dérivée nulle, soit :

$$\hat{\sigma}_m^2 = \frac{1}{N} \|TF_m(\underline{y}) - TF_m(\hat{x}_m)\|_2^2$$

On constate que l'on peut calculer cette expression directement par :

$$\hat{\sigma}_m^2 = \frac{1}{N} \sum_{k=K+1}^N TF_m(\underline{y})(k)^2$$

Cette formule indique que la variance estimée du bruit est la somme des densités spectrales de puissance du signal liées aux hautes fréquences.

ou encore :

$$\hat{\sigma}_m^2 = \frac{1}{N} \left(\|\underline{y}\|_2^2 - \sum_{k=1}^K TF_m(\underline{y})(k)^2 \right)$$

De même,

$$\hat{\sigma}_s^2 = \frac{1}{N} \|TF_s(\underline{y}) - TF_s(\hat{x}_s)\|_2^2$$

En remplaçant les variables par leurs estimées dans l'expression de la vraisemblance, on obtient :

$$L(\hat{x}_m, \hat{\sigma}_m; \underline{y}) = \frac{1}{(2\pi\hat{\sigma}_m)^{N/2}} \exp\left(-\frac{N}{2}\right)$$

De même,

$$L(\hat{x}_s, \hat{\sigma}_s; \underline{y}) = \frac{1}{(2\pi\hat{\sigma}_s)^{N/2}} \exp\left(-\frac{N}{2}\right)$$

On obtient alors le rapport de vraisemblance généralisé :

$$\frac{L(\hat{x}_m, \hat{\sigma}_m; \underline{y})}{L(\hat{x}_s, \hat{\sigma}_s; \underline{y})} = \left(\frac{\hat{\sigma}_s}{\hat{\sigma}_m} \right)^N$$

Cette formule indique que la classe la plus probable est celle pour laquelle on estime que le signal est le moins bruité.

Etant donné que K est faible, l'ordinateur calcule en pratique ce rapport de vraisemblance par la formule suivante :

$$\frac{L(\hat{x}_m, \hat{\sigma}_m; \underline{y})}{L(\hat{x}_s, \hat{\sigma}_s; \underline{y})} = \left(\frac{\|\underline{y}\|_2^2 - \sum_{k=1}^K TF_s(\underline{y})(k)^2}{\|\underline{y}\|_2^2 - \sum_{k=1}^K TF_m(\underline{y})(k)^2} \right)^{N/2}$$

Le calcul s'écrit simplement sous Matlab :

```
% rapport de vraisemblance
% N = 116 car il y a 116 régions.
ratio = ((norme - sum((F_M*y).^2))/(norme - sum((F_S*y).^2)))^(116/2);
```

Lorsque l'on choisit la **solution à probabilité d'erreur minimale** (par opposition à la solution bayésienne de coût minimal), on choisit d'attribuer à $\underline{y}$ la classe des malades lorsque :

$$\frac{L(\hat{x}_m, \hat{\sigma}_m; \underline{y})}{L(\hat{x}_s, \hat{\sigma}_s; \underline{y})} \geq 1$$

Cela peut se réécrire :

$$\sum_{k=1}^K TF_m(\underline{y})(k)^2 - TF_s(\underline{y})(k)^2 \geq 0$$

On retrouve ici la statistique de test évoquée dans l'équation (17) de l'article [1].

Ainsi, si la puissance du signal estimé sous l'hypothèse (H_m) est plus grande que celle du signal estimé sous l'hypothèse (H_s) , on considère que le sujet est malade.